

NetSentinel - Risk Scoring Pipeline

NetSentinel — Risk Scoring Pipeline

How an alert is classified, scored, and bucketed into High / Medium / Low — and exactly where each piece lives in the codebase.

1. The Full Pipeline

```
Network flow
|
STAGE 1 -- Is it an attack at all?          (binary Random Forest)
|
STAGE 2 -- If yes, what kind of attack?    (multiclass Random Forest)
|
STAGE 3 -- How risky is THIS alert?        (scoring formula)
|
Bucketed High / Medium / Low --> sorted into the SOC alert queue
```

Two separate models answer “is it an attack” and “what kind,” but neither of those alone tells you how urgently an analyst should care. That’s a third, deliberate step — Stage 3 — and it’s the part detailed below.

2. Stage 3 — What Determines High vs. Low

Factor 1 — Model Confidence

The binary model’s `predict_proba()` output — a 0-to-1 number for “how sure am I this is really an attack.” This measures **certainty**, not **danger**. A model can be 95% sure it just saw a port scan (low danger) and 80% sure about something that looks like infiltration (high danger) — confidence alone can’t separate those.

Factor 2 — Attack-Type Severity Weight

A fixed lookup table your team defines, based on how much damage each attack type represents **if it’s real**:

Attack type	Why this severity	Weight
Port Scan	Reconnaissance only — no breach yet	0.3
Web Attack	Could mean an exploited app, but often fails	0.6
Brute Force	Active attempt to gain access	0.7
DoS / DDoS	Service disruption, high visibility/impact	0.8
Botnet	Likely means a host is already compromised	0.9
Infiltration	Deepest stage — attacker is already inside	1.0

You should be able to defend these numbers out loud: *“Infiltration is weighted highest because it means compromise already happened; port scanning is weighted lowest because it’s just probing, not a breach.”*

Factor 3 — Repetition / Burst (planned enhancement, not yet built)

One flagged flow from an IP might be noise. Twenty flagged flows from the same IP in two minutes is a pattern. This factor counts alerts from the same source in a rolling time window and nudges the score up for coordinated-looking activity. **Flag this honestly as a stretch goal** if asked — it is not in the current baseline.

The Formula

```
risk_score      = model_confidence x severity_weight(attack_type)
risk_score_100  = round(risk_score x 100)
```

Bucketing

```
risk_score_100 >= 70      --> HIGH   (red)
40 <= risk_score_100 < 70 --> MEDIUM (amber)
risk_score_100 < 40      --> LOW    (grey)
```

Worked Examples

Attack type	Confidence	Calculation	Score	Bucket
Brute Force	0.88	0.88 x 0.7	62	MEDIUM
Infiltration	0.81	0.81 x 1.0	81	HIGH
Port Scan	0.95	0.95 x 0.3	28	LOW

Note the last row: highest model *certainty* of the three, but lowest *risk* — exactly why confidence alone can't drive ranking.

Where SHAP Fits

SHAP does **not** change the risk number. Its job is explaining *why* the model flagged the flow in the first place (e.g. “flagged mainly due to unusually high packet rate + short flow duration”). It rides alongside the risk score on the alert card — one number decides order, one explanation builds trust. Keep these two jobs separate in both your explanation and your code.

3. How Each Factor Gets Implemented in the Project

This is the part that turns the formula above into working code, and where it plugs into the architecture from the build plan.

3.1 Config — Severity Weights

A single source of truth, not scattered constants. One file, imported everywhere it's needed:

```
# ml_pipeline/severity_config.py

SEVERITY_WEIGHTS = {
    "PortScan": 0.3,
    "WebAttack": 0.6,
    "BruteForce": 0.7,
    "DoS": 0.8,
    "DDoS": 0.8,
    "Botnet": 0.9,
    "Infiltration": 1.0,
}

DEFAULT_WEIGHT = 0.5 # fallback for any label not in the table above

def get_severity_weight(attack_type: str) -> float:
    return SEVERITY_WEIGHTS.get(attack_type, DEFAULT_WEIGHT)
```

Keeping this as a plain dict (not hardcoded inside the scoring function) means you can tune weights after watching the demo dataset without touching any other code — and it's one file you can literally show a judge who asks “where do these numbers come from.”

3.2 Confidence — Pulled Straight From the Model

Inside the inference service, confidence is just the model's own probability output — no extra code needed beyond calling it correctly:

```
# api/scoring.py

def get_attack_confidence(binary_model, feature_vector) -> float:
    # predict_proba returns [P(normal), P(attack)]
    probabilities = binary_model.predict_proba([feature_vector])[0]
    return float(probabilities[1]) # P(attack)
```

3.3 The Risk Scoring Function — Where Factors 1 and 2 Combine

```
# api/scoring.py

from ml_pipeline.severity_config import get_severity_weight

def compute_risk(confidence: float, attack_type: str) -> dict:
    severity = get_severity_weight(attack_type)
    raw_score = confidence * severity
    score_100 = round(raw_score * 100)

    if score_100 >= 70:
        level = "HIGH"
    elif score_100 >= 40:
        level = "MEDIUM"
    else:
        level = "LOW"

    return {
        "risk_score": score_100,
        "risk_level": level,
        "confidence": round(confidence, 3),
        "severity_weight": severity,
    }
```

3.4 Plugging Into the `/alerts/score` Endpoint

This is where Stages 1-3 run back-to-back for a single flow, from the build plan's FastAPI service:

```
# api/main.py

@app.post("/alerts/score")
def score_flow(flow: FlowInput):
    features = extract_features(flow) # shared extractor

    # Stage 1 -- is it an attack?
    attack_conf = get_attack_confidence(binary_model, features)
    if attack_conf < ATTACK_THRESHOLD: # e.g. 0.5
        return {"status": "benign", "confidence": attack_conf}

    # Stage 2 -- what kind?
    attack_type = multiclass_model.predict([features])[0]

    # Stage 3 -- how risky?
    risk = compute_risk(attack_conf, attack_type)

    # Stage 4 -- explain (SHAP, separate from the score)
    top_features = explain_with_shap(binary_model, features)

    alert = save_alert(
        flow=flow,
        attack_type=attack_type,
        risk_score=risk["risk_score"],
        risk_level=risk["risk_level"],
        top_features=top_features,
```

```
)  
return alert
```

3.5 Database — Where Risk Gets Stored

Two columns added to the `alerts` table from the build plan, so the dashboard can sort/filter without recomputing anything:

```
ALTER TABLE alerts ADD COLUMN risk_score INTEGER; -- 0-100  
ALTER TABLE alerts ADD COLUMN risk_level TEXT; -- HIGH | MEDIUM | LOW
```

3.6 Dashboard — Where Risk Gets Used

The Alert Queue page sorts by `risk_score DESC` by default, and color-codes rows by `risk_level` (red / amber / grey) — this is the one-line payoff of the whole pipeline: an analyst opens the dashboard and the most dangerous thing is already at the top, not just the most recent thing.

3.7 Factor 3 (Burst) — If You Get to It

Implemented as a query, not a model change — counts recent alerts from the same source IP and adds a small bonus to the score:

```
def get_burst_bonus(src_ip: str, window_minutes: int = 5) -> int:  
    count = count_recent_alerts(src_ip, window_minutes)  
    if count >= 10:  
        return 15  
    elif count >= 5:  
        return 8  
    return 0
```

Add this to `risk_score` before bucketing, capped at 100. Build this only after Stages 1-3 are solid end-to-end — it's an enhancement, not a dependency.

4. Anticipated Question

“Why not just use the model’s raw confidence as risk?” Because confidence measures certainty, not consequence. A model can be very certain about something harmless. Multiplying by a severity weight is what turns “I’m sure” into “I’m sure, and this matters.”